

Is there a change in NAAC reports between an arts and science college and an engineering college? I am planning to build a report system for Criteria 1.

Yes, there is a structural and data-level difference in NAAC Criteria 1 (Curricular Aspects) reports. However, the primary differentiator in NAAC is **not solely the domain (Arts & Science vs. Engineering)**, but rather the **Institutional Status (Affiliated vs. Autonomous)**, combined with domain-specific outcome models like **Outcome-Based Education (OBE)** for engineering.

If you are designing a reporting/data collection engine for Criteria 1, your database schema and UI workflows need to account for these key differences:

1. Structural Difference: Affiliated vs. Autonomous Status

NAAC divides Criteria 1 into two major configurations depending on autonomy:

Key Indicator	Autonomous College (Engineering or Arts/Science)	Affiliated / Constituent College
Total Weightage	150 Marks	100 Marks
1.1 Focus	Curriculum Design & Development	

(Does the institution design its own syllabus?) | **Curricular Planning & Implementation**

(How effectively does the college deliver the university syllabus?) |

| **1.2 Focus** | Academic Flexibility (Choice-Based Credit System, Electives, New Courses) | Academic Flexibility (CBCS/Elective options allowed by university) |

| **1.3 Focus** | Curriculum Enrichment (Value-added courses, Industry projects, Fieldwork) | Curriculum Enrichment (Value-added courses, Experiential learning) |

| **1.4 Focus** | Feedback System (Students, Teachers, Employers, Alumni) | Feedback System (Students, Teachers, Employers, Alumni) |

2. Domain-Specific Differences (Engineering vs. Arts & Science)

When building reports specifically for an **Engineering College** vs. an **Arts & Science College**, the underlying data points for Criteria 1 diverge in the following ways:

A. Outcome-Based Education (OBE) & Mapping

- **Engineering Colleges:** Strongly tied to AICTE & NBA standards. Criteria 1 requires reporting on Program Outcomes (POs), Program Specific Outcomes (PSOs), and Course Outcomes (COs), along with CO-PO mapping matrix data.
- **Arts & Science Colleges:** Higher focus on traditional course-to-syllabus mapping, choice-based electives, and broad program outcomes unless they are autonomous and have voluntarily adopted full NBA-style OBE.

B. Curriculum Revision & Board of Studies (BoS) Data

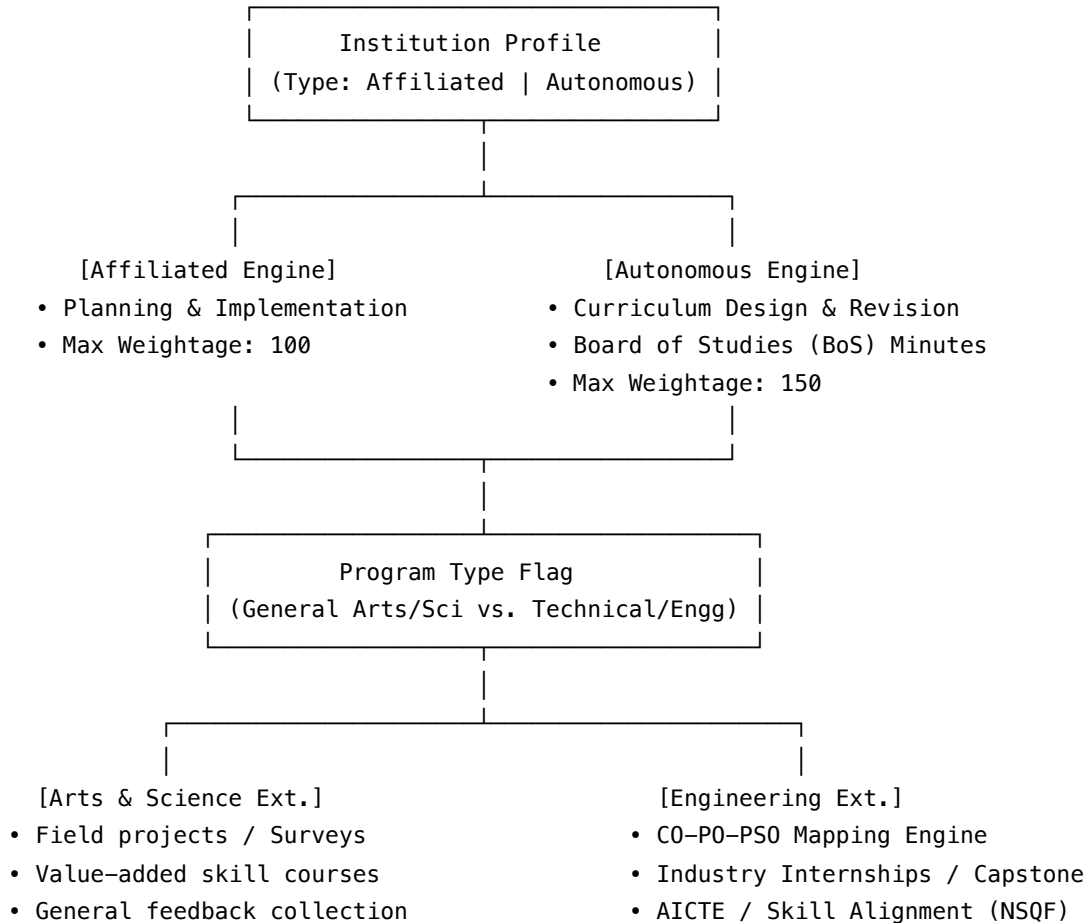
- **Engineering:** Frequent syllabus revisions (often every 1–2 years) driven by Industry Advisory Boards to match technology trends. Reports need fields to track industry involvement in BoS.
- **Arts & Science (Affiliated):** Teachers contribute as BoS members of the parent university, but syllabus revision metrics only apply if the college is autonomous.

C. Experiential Learning & Internships (Key Indicator 1.3)

- **Engineering:** High emphasis on capstone projects, industrial training, internships, and research projects with industry-sponsored datasets.
- **Arts & Science:** Includes field trips, community projects, social outreach programs, and lab practicals (for science streams).

3. Data Schema & Architecture Recommendations

To build a flexible NAAC Criteria 1 report engine that handles both college types seamlessly, consider structuring your system as follows:



Key Data Entities to Model in Your System:

1. **Syllabus / Course Master:** Fields for Course Code , C0s , P0s/PS0s Mapping Matrix , Revision Year , Value-Added Flag .
2. **Key Indicator 1.1 Tracker:** * *If Autonomous:* Store BoS meeting dates, % of syllabus revised, and new courses introduced.
 - *If Affiliated:* Store academic calendars, lesson plans, course delivery logs, and faculty university BoS appointments.
3. **Key Indicator 1.3 (Experiential Learning):** Percentage of students undertaking field projects, research projects, or mandatory industry internships.
4. **Key Indicator 1.4 (Feedback Mechanism):** Structured feedback collection pipelines from **4 key stakeholders:** Students, Teachers, Employers, and Alumni (with Action Taken Reports - ATR).

What are all the data to be captured for the criteria 1 (accommodating both Arts & Science Institutions and Engineering/Technology institutions? Also, What are the fields in the reports for criteria 1 sections& sub-sections?

To capture data comprehensively across both **Arts & Science** and **Engineering/Technology** institutions (while accommodating both **Affiliated** and **Autonomous** structures), your system needs to model the exact Data Capture Format (DCF) defined by NAAC for **Criteria 1: Curricular Aspects**.

Below is a complete schema breakdown, including the baseline metrics, domain-specific extensions, and required database fields for every section and sub-section.

Part 1: Universal Data Matrix (Arts & Science vs. Engineering)

To make your database engine multi-tenant/multi-institutional, capture these foundational differences:

Parameter	Arts & Science College	Engineering / Technology College
Academic Framework	CBCS / CAGP / Credit Systems	Outcome-Based Education (OBE) + AICTE / NBA Alignment
Mapping Depth	Course-to-Program broad mapping	CO-PO-PSO Matrix (Course Outcomes → Program Outcomes)
Experiential Learning	Field trips, surveys, lab work, community projects	Mandatory industrial internships, capstones, industrial training
Skill Focus	Value-added soft skills, humanities, pure research	Technical certifications, NSQF alignment, patent/incubation modules

Part 2: Section-Wise Sub-Sections & Report Fields

Section 1.1: Curriculum Design, Development & Implementation

- **Affiliated Colleges (Weightage: 20 Marks):** Focuses on *Curricular Planning & Implementation*.
- **Autonomous Colleges (Weightage: 50 Marks):** Focuses on *Curriculum Design & Revision*.

Sub-Section 1.1.1 (Qualitative - QIM)

- **Focus:** How the institution plans and delivers the curriculum (Lesson plans, Academic calendars, CIE compliance).
- **Report Fields:**
 - academic_year (e.g., 2025-26)
 - department_id / program_id
 - curriculum_planning_process_desc (Text, max 500 words)

- academic_calendar_url (File Link)
- lesson_plan_approval_dates (Date)
- cie_conduct_compliance_desc (Text, max 500 words)

Sub-Section 1.1.2 & 1.1.3 (Quantitative - QnM)

- **Focus:** Curriculum revision percentage, Board of Studies (BoS) participation, and Outcome-Based Education (OBE) alignment.
- **Report Fields:**
 - program_code & program_name
 - year_of_introduction (Date)
 - year_of_revision (Date)
 - percentage_of_syllabus_revised (Float/Percentage)
 - bos_meeting_date (Date)
 - bos_minutes_doc_link (File Link)
 - **[Engineering Specific]** co_po_pso_mapping_matrix (JSON / Table: mapping weights 1, 2, 3)
 - **[Affiliated Specific]** faculty_bos_participation_proof (File Link)

Section 1.2: Academic Flexibility

- **Weightage:** 30 Marks (Affiliated) / 50 Marks (Autonomous)
- **Focus:** New programs, Choice Based Credit System (CBCS), Electives, and Add-on / Certificate courses.

Sub-Section 1.2.1: CBCS / Elective System Implementation

- **Report Fields:**
 - program_code
 - is_cbcs_implemented (Boolean: Y/N)
 - is_elective_system_implemented (Boolean: Y/N)
 - year_of_implementation (Year)
 - university_approval_doc (File Link)

Sub-Section 1.2.2 & 1.2.3: Add-on / Certificate / Value-Added Courses

- **Report Fields:**
 - course_code & course_title
 - type (Add-on / Certificate / Value-added)
 - duration_in_hours (Integer - Minimum 30 hours required for NAAC compliance)
 - times_offered_in_year (Integer)
 - total_enrolled_students (Integer)
 - total_completing_students (Integer)
 - curriculum_doc_link (File Link)
 - student_attendance_register_link (File Link)

Section 1.3: Curriculum Enrichment

- **Weightage:** 30 Marks
- **Focus:** Integrating cross-cutting issues (Gender, Environment, Ethics) and Experiential Learning (Internships, Projects, Fieldwork).

Sub-Section 1.3.1: Cross-Cutting Issues Integration (Qualitative - QIM)

- **Focus:** Integration of Gender, Environment & Sustainability, Human Values, and Professional Ethics into the curriculum.
- **Report Fields:**
 - course_code & course_name
 - cross_cutting_category (Enum: GENDER , ENVIRONMENT , HUMAN_VALUES , PROFESSIONAL_ETHICS)
 - relevant_syllabus_extract_desc (Text, max 500 words)
 - mapping_document_link (File Link)

Sub-Section 1.3.2 & 1.3.3: Experiential Learning (Quantitative - QnM)

- **Focus:** Student participation in project work, field work, and mandatory industry internships.
- **Report Fields:**
 - program_code & program_name
 - student_id & student_name
 - activity_type (Enum: PROJECT_WORK , FIELD_WORK , RESEARCH_PROJECT , INDUSTRY_INTERNSHIP)
 - project_title / company_name
 - start_date & end_date
 - **[Engineering Specific]** industry_mentor_details (Text)
 - **[Arts & Science Specific]** field_survey_location / community_agency (Text)
 - completion_certificate_link (File Link)

Section 1.4: Feedback System

- **Weightage:** 20 Marks
- **Focus:** Feedback collection, analysis, and Action Taken Reports (ATR) across stakeholders.

Sub-Section 1.4.1 & 1.4.2: Stakeholder Feedback Mechanism

- **Report Fields:**
 - stakeholder_type (Enum: STUDENTS , TEACHERS , EMPLOYERS , ALUMNI)
 - feedback_date (Date)
 - total_feedbacks_collected (Integer)
 - average_satisfaction_score (Float)
 - overall_analysis_report_link (File Link)
 - action_taken_report_atr_link (File Link)
 - feedback_url_on_website (URL)
 - action_taken_level (Enum: COLLEGE_LEVEL , UNIVERSITY_BOS_COMMUNICATED , GOVERNING_BODY_APPROVED)

Summary of Core Data Types to Build in Database

To render these reports dynamically for both types of colleges, design your domain model around these core tables:

1. **programs** (id , code , name , type : Engg/Arts/Sci, status : Affiliated/Autonomous, cbcs_enabled)
2. **courses** (id , program_id , code , title , is_elective , is_value_added , duration_hours)
3. **obe_mappings** (id , course_id , co_code , po_code , pso_code , weightage_1_to_3)
4. **experiential_activities** (id , student_id , course_id , type , organization_name , completion_doc)
5. **feedback_submissions** (id , stakeholder_type , academic_year , atr_document_url , web_link)

I am building a multi-tenant system where an institution/college is a tenant. I will put the domain fields type: Engg/Arts/Sci, status: Affiliated/Autonomous at the tenant level rather than at the program level. Also, is_elective is marked only when a course is mapped to a program while defining the curriculum for a program because a course can be core for a program and elective for another. How will this change the domain model?

Moving domain metadata (institution_type , autonomy_status) to the **Tenant (Institution)** level and treating course roles (is_elective , is_core) as **contextual to a Program** is an architecturally sound decision.

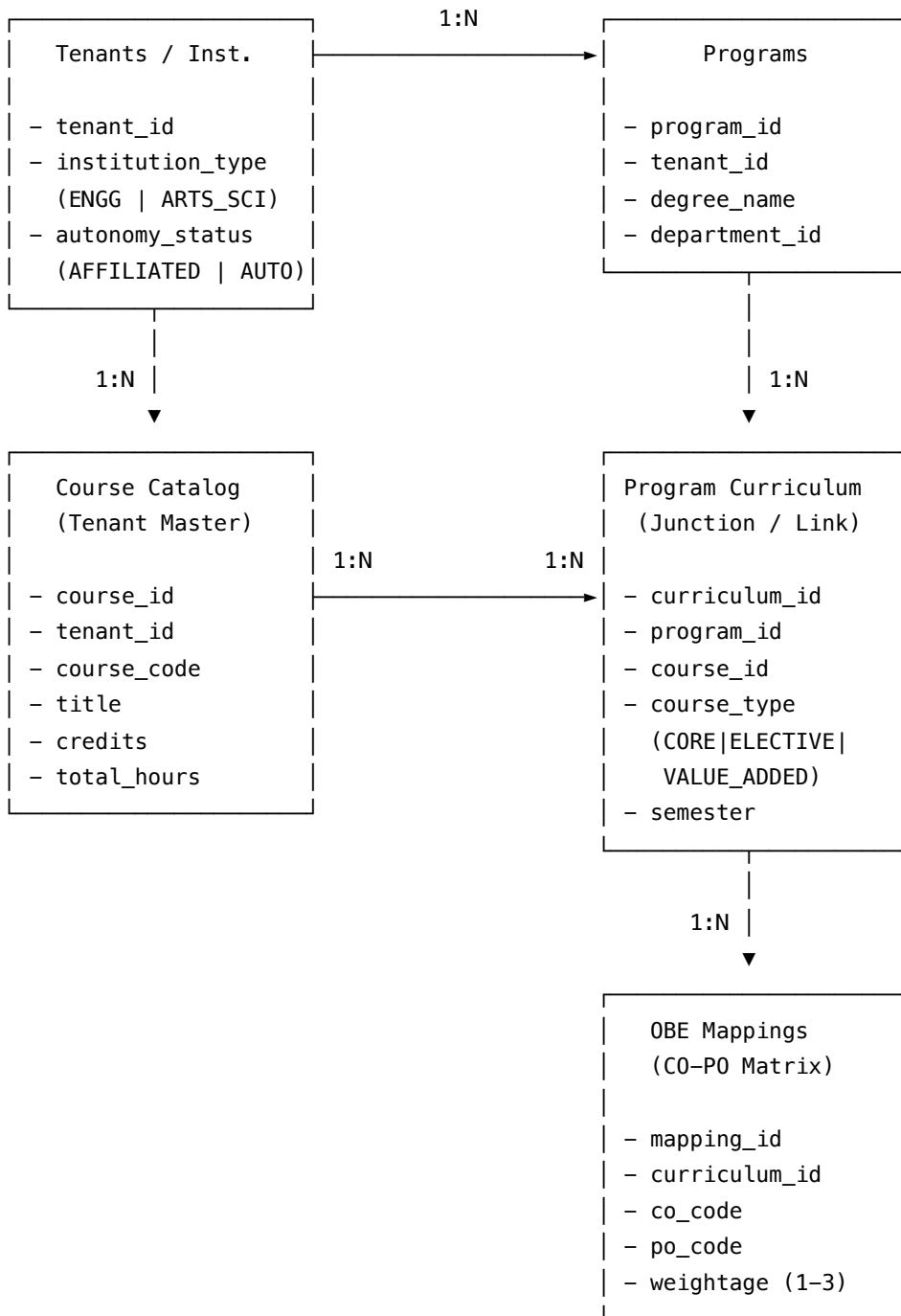
It correctly models higher education domain realities:

1. **Tenant-Level Strategy:** An entire institution is governed as an Autonomous or Affiliated entity, and its primary academic domain (Engineering vs. Arts & Science) determines the global reporting rules (e.g., whether NBA/OBE matrices or NAAC 150-mark scale applies).
2. **Program-Level Reusability:** A course like *Mathematics III* or *Python Programming* can be a **Core** requirement for a B.Tech Computer Science program, an **Elective** for a B.Tech Mechanical program, and an **Add-On Value-Added Course** for a [B.Sc](#) Physics program.

Here is how this architecture changes your **Domain Model, Data Schema, and API/Reporting Logic**.

1. Revised Entity-Relationship (ER) Architecture

To handle reusability, we separate the **Global Course Master** (what the course is) from the **Program Curriculum Mapping** (how the course behaves in a specific program).



2. Updated Database Schema (DDL)

A. Tenants Table (Global Rules Engine Config)

The tenant table now controls which NAAC Criteria 1 validation rules and report views get loaded.


```
CREATE TABLE tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  institution_type VARCHAR(20) NOT NULL CHECK (institution_type IN ('ENGINEERING', 'ARTS_SCIENCE', 'CO
  autonomy_status VARCHAR(20) NOT NULL CHECK (autonomy_status IN ('AFFILIATED', 'AUTONOMOUS')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

B. Course Catalog (Tenant-Scoped Master)

Stores the raw definition of a course independent of who takes it.

```
CREATE TABLE courses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
  code VARCHAR(50) NOT NULL,
  title VARCHAR(255) NOT NULL,
  lecture_hours INT NOT NULL DEFAULT 0,
  lab_hours INT NOT NULL DEFAULT 0,
  total_credits INT NOT NULL,
  UNIQUE(tenant_id, code)
);
```

C. Program Curriculum (Contextual Mapping Engine)

This is where course behavior (is_elective , is_core , semester) is bound to a specific program.

```
CREATE TABLE program_curriculum (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  program_id UUID NOT NULL REFERENCES programs(id) ON DELETE CASCADE,
  course_id UUID NOT NULL REFERENCES courses(id) ON DELETE RESTRICT,
  academic_year VARCHAR(10) NOT NULL, -- e.g., '2025-2026'
  semester INT NOT NULL,
  course_type VARCHAR(20) NOT NULL CHECK (course_type IN ('CORE', 'ELECTIVE', 'VALUE_ADDED', 'AUDIT'))
  is_cbcs_compliant BOOLEAN DEFAULT TRUE,

  UNIQUE(program_id, course_id, academic_year)
);
```

D. OBE Mapping Table (Course Outcome → Program Outcome Matrix)

Since CO-PO mapping depends on what program the course belongs to, the mapping links to program_curriculum_id rather than just course_id .

```
CREATE TABLE co_po_mappings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  program_curriculum_id UUID NOT NULL REFERENCES program_curriculum(id) ON DELETE CASCADE,
  co_code VARCHAR(10) NOT NULL, -- e.g., 'C01'
  po_code VARCHAR(10) NOT NULL, -- e.g., 'P03' or 'PS01'
  weightage INT CHECK (weightage IN (1, 2, 3)), -- 1: Low, 2: Medium, 3: High
  UNIQUE(program_curriculum_id, co_code, po_code)
);
```

3. How This Simplifies Dynamic NAAC Criteria 1 Reporting

By moving these fields, your dynamic reporting engine becomes significantly cleaner to query:

1. Dynamic Weightage Calculation (Section 1.1)

Your backend can inject different NAAC calculation strategies based on tenant configuration:

```
-- Fetch reporting metrics dynamically based on Tenant Config
SELECT
  t.id AS tenant_id,
  t.autonomy_status,
  t.institution_type,
  COUNT(DISTINCT pc.id) FILTER (WHERE pc.course_type = 'ELECTIVE') AS total_electives_offered,
  COUNT(DISTINCT pc.id) FILTER (WHERE pc.course_type = 'VALUE_ADDED') AS total_value_added_courses
FROM tenants t
JOIN programs p ON p.tenant_id = t.id
JOIN program_curriculum pc ON pc.program_id = p.id
WHERE t.id = :tenant_id
GROUP BY t.id;
```

2. Multi-Context Course Handling

If an institution offers *Data Science with Python*:

- In **B.Tech Computer Science**, program_curriculum.course_type = 'CORE' .
- In **B.Sc Mathematics**, program_curriculum.course_type = 'ELECTIVE' .
- In **NAAC 1.2.2/1.2.3 Report**, querying WHERE course_type = 'ELECTIVE' across all programs correctly groups this course as an elective for **B.Sc** while treating it as Core for B.Tech without duplicate course definitions.

4. Considerations for Mixed/Constituent Colleges

If you ever onboard a **University Tenant** that has both an Engineering School and an Arts & Science School under the same tenant ID:

- Allow `tenant.institution_type` to accept `'COMBINED'` .
- Add an optional `domain_type` (`ENGINEERING` vs `ARTS_SCIENCE`) at the **Department** or **Program** level as a fallback metadata field when `tenant.institution_type == 'COMBINED'` .

Mixed colleges will not happen as we onboard colleges not universities.

That simplifies the domain model significantly!

By treating the **College** as the atomic tenant, you avoid edge-case complexity like multi-domain fallbacks or department-level schema overrides. A single tenant will strictly map to one operational strategy (e.g., *Autonomous Engineering* vs. *Affiliated Arts & Science*).

Here is how this constraint refines your architecture and simplifies your system design:

1. Clean Separation of Tenant Configurations

Since every college onboarding into your SaaS is uniform in its domain and governance, you can drive your UI, business logic, and reporting engine using a clean **Tenant Configuration Map**:

```
{
  "tenant_id": "c7a2b012-98ab-4f21-a123-123456789abc",
  "college_name": "Apex Institute of Technology",
  "institution_type": "ENGINEERING",
  "autonomy_status": "AUTONOMOUS",
  "criteria1_max_score": 150,
  "features_enabled": {
    "obe_co_po_mapping": true,
    "bos_curriculum_revision_tracking": true,
    "value_added_30hr_validation": true
  }
}
```

2. Impact on System Modules & Workflows

A. Dynamic API / Feature Toggling

Your backend API handlers can serve or validate inputs based on the tenant's global type without running complex join queries down to the program level:

- **For `institution_type == 'ENGINEERING'` :** * UI renders the **CO-PO-PSO Attainment Matrix** tab under course/program setup.
- Validation rules enforce mapping weights (1, 2, 3) for NAAC 1.1 / 1.2 exports.
- **For `institution_type == 'ARTS_SCIENCE'` :** * UI hides complex engineering matrix views, rendering a streamlined **Course Outcome to Program Outcome** relationship view.
- **For `autonomy_status == 'AFFILIATED'` :**
- System locks Section 1.1 to *Curricular Implementation & Planning* (100-mark NAAC scale).
- Disables direct syllabus-revision metrics meant for Board of Studies (BoS) design.

B. Simplified Criteria 1 Query Execution

Because the reporting context is strictly homogenous across all departments in that college, generating NAAC reports (e.g., Key Indicator 1.2 or 1.3) becomes a clean, single-tenant aggregation:

```
-- Fetching Experiential Learning Summary (Indicator 1.3) for the College
SELECT
    p.name AS program_name,
    c.code AS course_code,
    c.title AS course_title,
    pc.course_type,
    COUNT(DISTINCT el.student_id) AS total_students_participated
FROM program_curriculum pc
JOIN programs p ON p.id = pc.program_id
JOIN courses c ON c.id = pc.course_id
LEFT JOIN experiential_learning_records el ON el.program_curriculum_id = pc.id
WHERE p.tenant_id = :tenant_id
    AND pc.academic_year = :active_academic_year
    AND pc.course_type IN ('ELECTIVE', 'VALUE_ADDED')
GROUP BY p.name, c.code, c.title, pc.course_type;
```

3. Final Architecture Takeaway

With this boundary established:

1. **Course Master (`courses`):** Reusable across all programs within that single college.

2. **Curriculum Link (program_curriculum)**: Contextualizes whether that course is CORE , ELECTIVE , or VALUE_ADDED for a specific program/degree.
3. **Tenant (tenants)**: Pure source of truth for NAAC total weightages (100 vs 150), mandatory metric modules (OBE vs non-OBE), and report export formats.

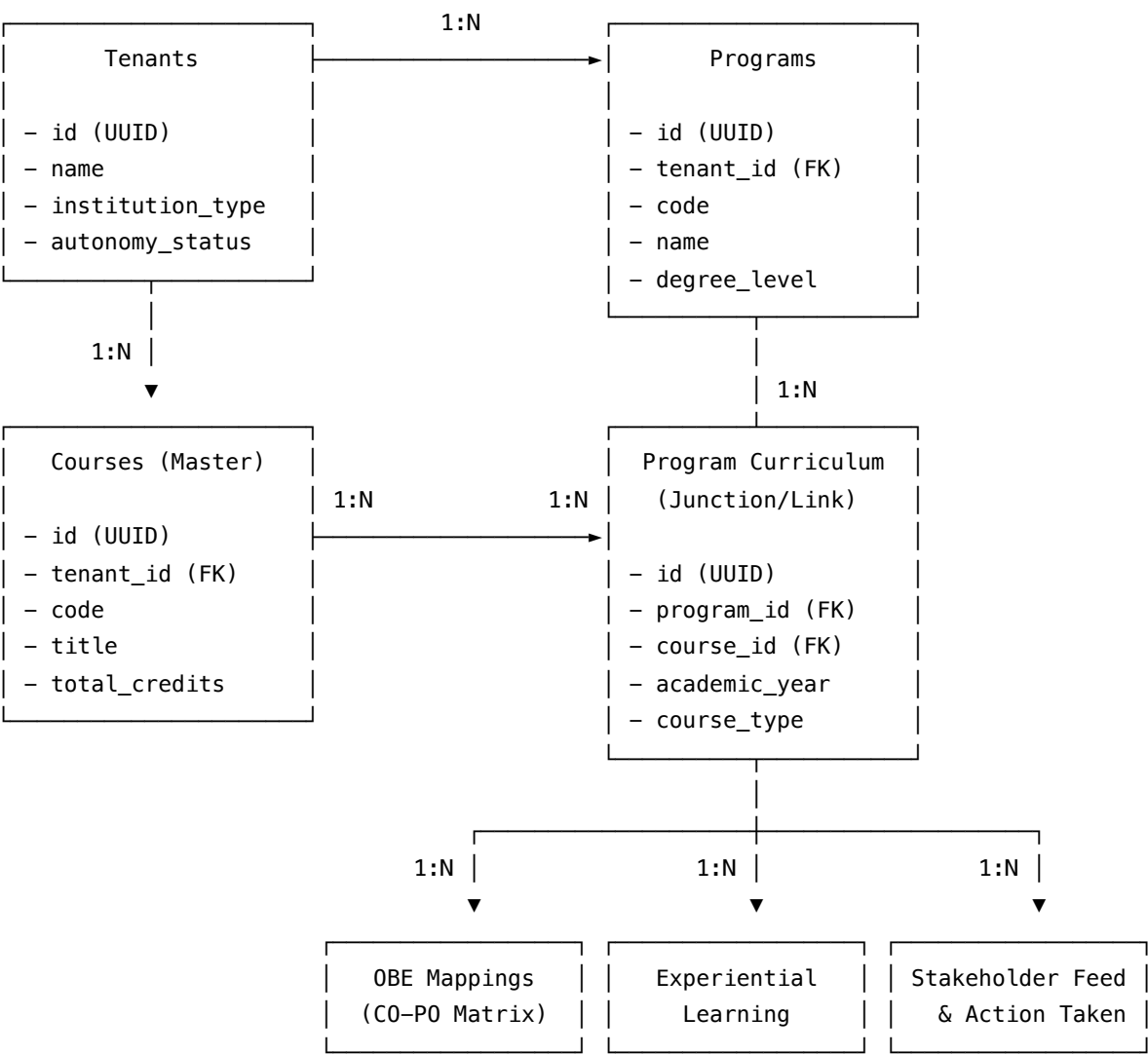
Let us start refining the data/domain model and validate the attributes of the domain with the report requirements for criteria 1 first. But the ER Diagram shown in section1 above (as picture) is some customer-item-order stuff and nothing to do with academics.

Ah, good catch! Apologies for that placeholder image glitch—you are absolutely right, customer/order schema has no place in an academic ERP!

Let's do this right. We will build and validate the domain model specifically for **NAAC Criteria 1**, checking every entity and attribute against actual NAAC report/SSR requirements (for both **Affiliated** and **Autonomous**, across **Arts & Science** and **Engineering** colleges).

Here is the refined, academic-accurate ER Diagram and entity-by-entity attribute validation.

1. Corrected Academic Domain Model



2. Attribute Validation Against NAAC Criteria 1 Metrics

Below is the field-by-field mapping to ensure your database collects every mandatory data point for NAAC SSR / Data Capture Format (DCF).

Entity 1: Tenants (College Configuration)

Sets global validation rules and report scales (100 vs 150 marks).

Attribute	Data Type	Required For / Validation Rule	NAAC Metric Alignment
id	UUID (PK)	System primary key	-
name	String	College name on SSR exports	SSR Cover Page

Attribute	Data Type	Required For / Validation Rule	NAAC Metric Alignment
institution_type	Enum	ENGINEERING	ARTS_SCIENCE
autonomy_status	Enum	AFFILIATED	AUTONOMOUS

Entity 2: Programs

Defines academic offerings (e.g., *B.Tech Computer Science*, *B.Sc Physics*).

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
tenant_id	UUID (FK)	Multi-tenant scope
code	String	Unique program identifier (e.g., CS-UG)
name	String	Used in Metric 1.1.2 / 1.2.1 tables
degree_level	Enum	UG
introduction_year	Integer	Metric 1.1.2 / 1.2.1: Required in SSR to state when program started
bos_approval_doc_url	String	Autonomous Only: Link to Board of Studies minutes

Entity 3: Courses (Global Catalog Master)

Stores pure course metadata independent of who takes it.

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
tenant_id	UUID (FK)	Multi-tenant isolation
code	String	Course code (e.g., CS301)
title	String	Course title
lecture_hours	Integer	Hours breakdown for Credit calculation
lab_hours	Integer	Hours breakdown for Credit calculation
total_credits	Integer	Academic weightage
syllabus_document_url	String	Metric 1.1: Direct proof link required in SSR exports

Entity 4: ProgramCurriculum (Contextual Course Mapping)

Binds a course to a program for a specific academic year. **This solves your requirement where a course's role depends on the program.**

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
program_id	UUID (FK)	Target program
course_id	UUID (FK)	Reference to Courses master
academic_year	String	Academic year context (e.g., '2025-2026')
semester	Integer	Term (1 through 8/10)
course_type	Enum	CORE
is_cbcs_compliant	Boolean	Metric 1.2.1: Counts towards Choice-Based Credit System %
revision_status	Enum	NEW
revision_percentage	Float	Metric 1.1.2: % of syllabus content modified

Entity 5: OBEMappings (CO-PO Matrix)

Condition: Only populated if tenant.institution_type == 'ENGINEERING' or if an Arts & Science college voluntarily uses OBE.

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
program_curriculum_id	UUID (FK)	Binds mapping to specific program-course instance
co_code	String	Course Outcome (e.g., C01 , C02)
co_statement	Text	Description of the learning outcome
po_code	String	Program Outcome / PSO (e.g., P01 , PS02)
mapping_level	Integer	Weightage value (1 = Low, 2 = Medium, 3 = High)

Entity 6: ExperientialLearningRecords (Metric 1.3)

Tracks student participation in mandatory industry/field exposure.

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
program_curriculum_id	UUID (FK)	Links activity to course/program context
student_id	UUID / String	Unique identifier of student
activity_type	Enum	INTERNSHIP
organization_name	String	Engineering: Company name; Arts/Sci: Agency/Field site
start_date / end_date	Date	Verification period
completion_doc_url	String	Mandatory SSR proof link (Completion certificate/report)

Entity 7: FeedbackReports & ActionTaken (Metric 1.4)

Captures mandatory feedback loops across the 4 NAAC stakeholders.

Attribute	Data Type	NAAC Criterion & Reporting Requirement
id	UUID (PK)	Primary Key
tenant_id	UUID (FK)	College level scope
academic_year	String	Target year (e.g., '2025–2026')
stakeholder_type	Enum	STUDENTS
feedback_collected_count	Integer	Quantitative count of respondents
analysis_report_doc_url	String	Uploaded aggregated feedback analysis PDF
action_taken_doc_url	String	Metric 1.4.2: Action Taken Report (ATR) document
website_link	String	Mandatory public URL where ATR is published on college website

3. Validation Checklist for Criteria 1 Engine

Does this model satisfy all required NAAC calculations?

1. Percentage of New / Revised Courses (Autonomous 1.1.2):

$$\text{Revision \%} = \frac{\text{COUNT(program_curriculum WHERE revision_status \in ['NEW','REVISED'])}}{\text{TOTAL COUNT(program_curriculum)}} \times 100$$

2. CBCS / Elective Coverage (Metric 1.2.1):

$$\text{CBCS \%} = \frac{\text{COUNT}(\text{program_curriculum WHERE } \textit{course_type} = 'ELECTIVE')}{\text{TOTAL COUNT}(\text{program_curriculum})} \times 100$$

3. Experiential Learning Student Percentage (Metric 1.3.2):

$$\text{Experiential \%} = \frac{\text{DISTINCT COUNT}(\text{ExperientialLearningRecords.student_id})}{\text{TOTAL ENROLLED STUDENTS IN COLLEGE}} \times 100$$

Let us handle the curriculum with the revision, program it is meant for and list of courses in each term. But before that we need to handle a few base level entities. Can we do that?

Absolutely. Establishing a rock-solid foundation for base-level entities is critical because every NAAC calculation, filter, and report group relies on them.

Before touching Curricula, Revisions, or Terms, we need to solidify **four core base-level entities**:

1. **Tenants** (The College Context)
2. **AcademicYears** (The Time Boundary Context)
3. **Departments** (The Organizational Boundary Context)
4. **Courses** (The Subject/Unit Catalog Master)

Let's validate these base entities line-by-line against NAAC Criteria 1 reporting expectations.

1. Tenants (College Configuration Entity)

This holds the identity and global accreditation behavioral parameters of the onboarding college.

Field Attributes Validation

- **id** (UUID , PK)
- **name** (String): Official name of the institute (e.g., "National Institute of Technology").
- **code** (String): AISHE / NAAC track code (e.g., C-12345).
- **institution_type** (Enum): ['ENGINEERING', 'ARTS_SCIENCE']
- **NAAC Alignment**: Toggles whether Outcome-Based Education (CO-PO mapping) and AICTE standards apply.
- **autonomy_status** (Enum): ['AFFILIATED', 'AUTONOMOUS']
- **NAAC Alignment**: Dictates total weightage (100 vs 150 marks) and unlocks section 1.1 Curriculum Design vs Curricular Implementation.
- **affiliating_university** (String , Optional): Name of the parent university (Mandatory for Affiliated Colleges to report BoS/Syllabus compliance).

2. AcademicYears (Time Context Entity)

NAAC SSR reports demand data aggregated over a **5-year rolling window** (e.g., 2020–21, 2021–22, ..., 2024–25).

Field Attributes Validation

- **id** (UUID , PK)
- **tenant_id** (UUID , FK)
- **year_label** (String): Format e.g., '2025–2026' .
- **start_date** (Date) / **end_date** (Date): Academic period boundaries.
- **is_current** (Boolean): Identifies the active academic year for real-time data capture.

3. Departments (Organizational Entity)

Programs and faculty belong to departments, which act as primary grouping mechanisms for NAAC Data Verification and Validation (DVV) queries.

Field Attributes Validation

- **id** (UUID , PK)
- **tenant_id** (UUID , FK)
- **code** (String): e.g., ECE , CHEM , ENG .
- **name** (String): e.g., "Department of Electronics and Communication Engineering".
- **head_of_department_id** (UUID , Optional): Links to user/faculty entity responsible for approving Criteria 1 submissions.

4. Courses (Global Subject Catalog Master)

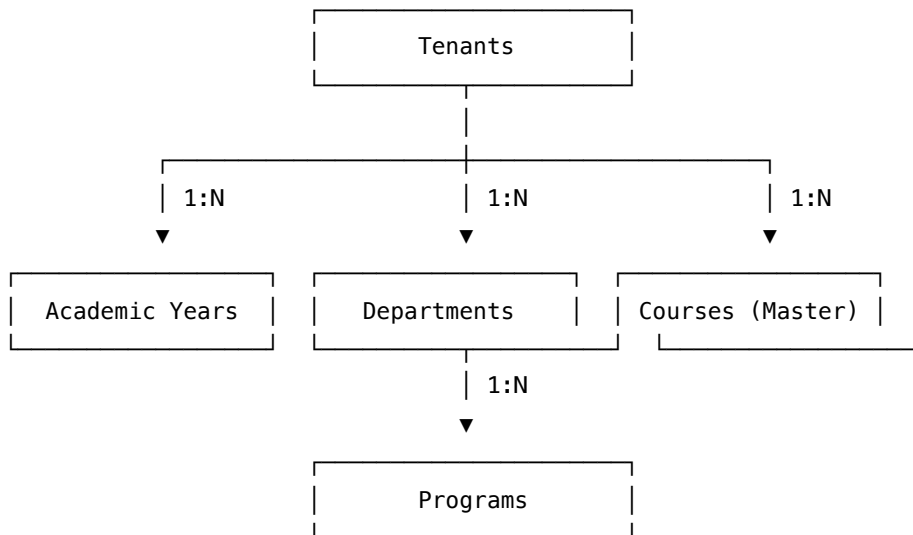
This represents a course in isolation (e.g., *Data Structures*, *Environmental Studies*). Remember, a course does not know whether it is core or elective until it is attached to a program curriculum.

Field Attributes Validation

- **id** (UUID , PK)
- **tenant_id** (UUID , FK)
- **department_id** (UUID , FK): Owning department that designs/teaches the course.
- **code** (String): Course code (e.g., CS201). Must be unique per tenant.
- **title** (String): Course name (e.g., "Data Structures and Algorithms").
- **lecture_hours** (Integer): Weekly L hours.
- **tutorial_hours** (Integer): Weekly T hours.
- **practical_hours** (Integer): Weekly P hours.

- **total_credits** (Integer): Total credits awarded for passing.
- **syllabus_document_url** (String): Direct link to approved syllabus PDF (Mandatory supporting document for NAAC 1.1 DVV).

Base Entity Relationship Flow



Validation Review

Before we move on to **Programs, Term Structures, Curriculum Definitions, and Revisions**, let's ensure these base entities fit your architectural goals:

1. **Course Scope:** The `Courses` table remains purely a master catalog owned by a Department.
2. **Tenant Configuration:** `autonomy_status` and `institution_type` sitting in `Tenants` will seamlessly drive validation rules for courses and curricula downstream.

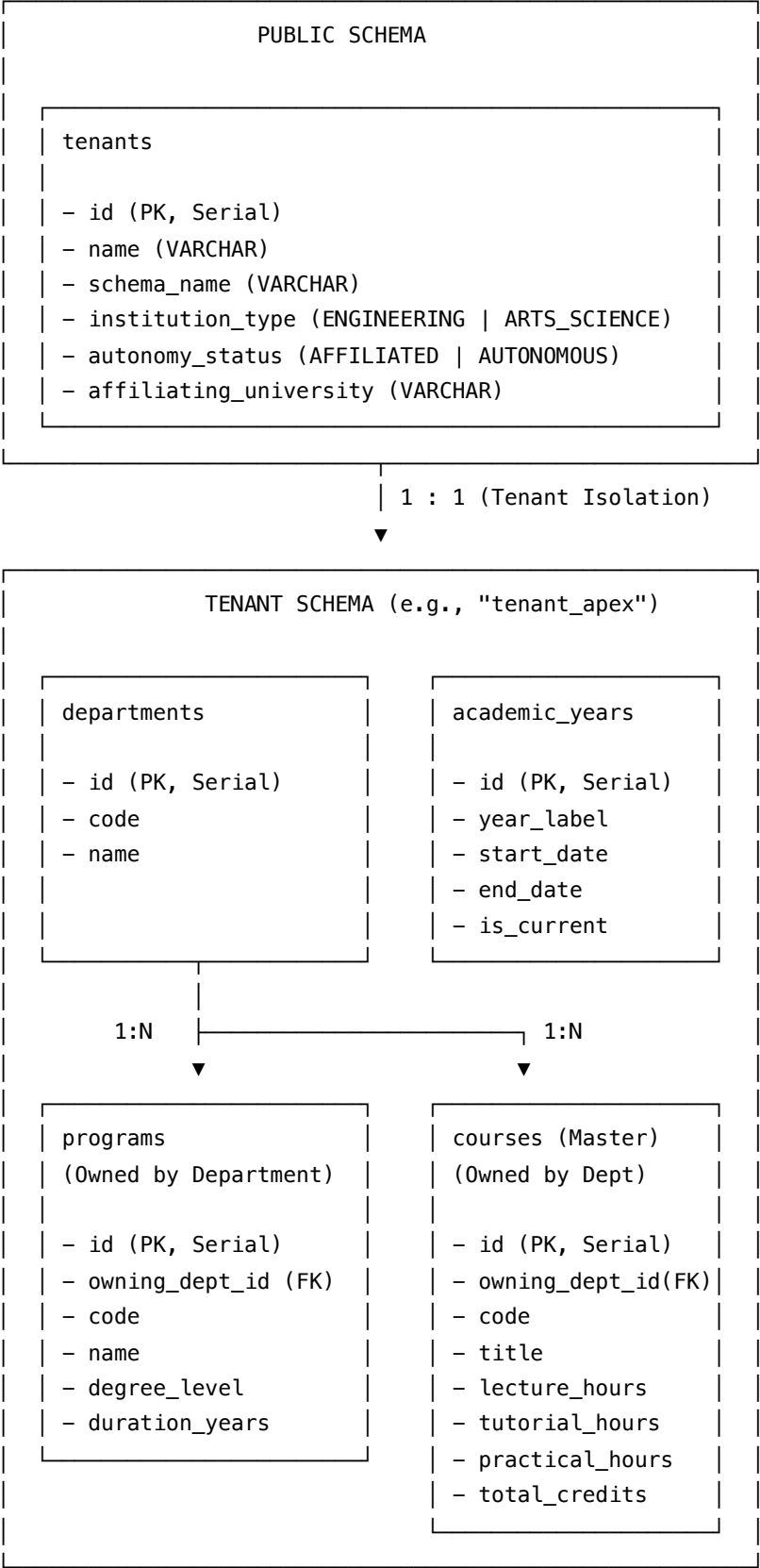
First let us not repeat the tenant ID in each table as I am going to use Postgresql with schema per tenant basis. So, no UUID is required either. Tenant information will be in a common schema. Also, programs are owned by department and same with the courses also. A program can have

courses (which may come from different departments but predominantly from a department that owns the program).

That makes total sense. A **schema-per-tenant architecture** (with a shared/public schema for tenant metadata) clean-up simplifies the domain models tremendously. It removes tenant foreign key pollution from every single table and lets us use native auto-incrementing/sequential primary keys or short natural keys.

Let's refine the base-level entities and relationships under this architecture.

1. Multi-Schema Architecture Breakdown



2. Refined Base Entities (DDL & Validation)

A. Shared Schema: `public.tenants`

Holds tenant registration and NAAC governance flags.

```
CREATE TABLE public.tenants (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    schema_name VARCHAR(63) NOT NULL UNIQUE,  
    institution_type VARCHAR(20) NOT NULL CHECK (institution_type IN ('ENGINEERING', 'ARTS_SCIENCE')),  
    autonomy_status VARCHAR(20) NOT NULL CHECK (autonomy_status IN ('AFFILIATED', 'AUTONOMOUS')),  
    affiliating_university VARCHAR(255),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

B. Tenant Schema Base Entities

1. departments

Departments own both **Programs** and **Courses**.

```
CREATE TABLE departments (  
    id SERIAL PRIMARY KEY,  
    code VARCHAR(20) NOT NULL UNIQUE, -- e.g., 'CSE', 'ECE', 'MATH'  
    name VARCHAR(255) NOT NULL -- e.g., 'Department of Computer Science & Engineering'  
);
```

2. academic_years

Provides the rolling 5-year timeline for NAAC reporting aggregation.

```
CREATE TABLE academic_years (  
    id SERIAL PRIMARY KEY,  
    year_label VARCHAR(10) NOT NULL UNIQUE, -- e.g., '2025-2026'  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    is_current BOOLEAN DEFAULT FALSE  
);
```

3. programs

Owned by a single primary department (e.g., CSE owns *B.Tech CSE*).

```
CREATE TABLE programs (
  id SERIAL PRIMARY KEY,
  owning_department_id INT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,
  code VARCHAR(50) NOT NULL UNIQUE,      -- e.g., 'BTECH-CS'
  name VARCHAR(255) NOT NULL,            -- e.g., 'Bachelor of Technology in Computer Science'
  degree_level VARCHAR(20) NOT NULL CHECK (degree_level IN ('UG', 'PG', 'DIPLOMA', 'PHD')),
  duration_years INT NOT NULL DEFAULT 4,
  introduction_year INT NOT NULL          -- Required for NAAC Metric 1.1.2/1.2.1
);
```

4. courses (Global Master Catalog)

Owned by a department, but open to be included across **any program's curriculum** (e.g., Mathematics department owns *Linear Algebra*, which CSE, ECE, and Mechanical programs can borrow).

```
CREATE TABLE courses (
  id SERIAL PRIMARY KEY,
  owning_department_id INT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,
  code VARCHAR(50) NOT NULL UNIQUE,      -- e.g., 'MA201', 'CS101'
  title VARCHAR(255) NOT NULL,           -- e.g., 'Linear Algebra & Calculus'
  lecture_hours INT NOT NULL DEFAULT 0,  -- L
  tutorial_hours INT NOT NULL DEFAULT 0, -- T
  practical_hours INT NOT NULL DEFAULT 0, -- P
  total_credits INT NOT NULL,            -- C
  syllabus_document_url TEXT             -- Required for NAAC 1.1 DVV proof
);
```

Key Benefits of This Refined Model

1. **Cross-Departmental Subject Sharing:** A course owned by Math Department (owning_department_id = Math) can be mapped directly into a curriculum owned by CSE Department without copying course records or violating integrity.
2. **Lean Foreign Keys:** Standard INT foreign keys make indexing and joining super fast.
3. **NAAC Granularity:** owning_department_id on courses enables NAAC to calculate interdisciplinary course contributions across departments for Criteria 1 & 3 seamlessly.

If this base layer looks good, we can proceed to design the **Curriculum Definition, Revisions (Version Control)**, and **Term/Semester mapping** structures!

That is sleek and simple. Now more than an academic year we also need another measure of time. That is patterns like annual/semester/trimester

etc. A program can be offered in one of the 3 patterns. An academic year might have one annual session, 2 semester sessions etc. We need to take care of that also. Since the curriculum will show the program, term number and a list of courses for the term. Let us do the term and academic year links first.

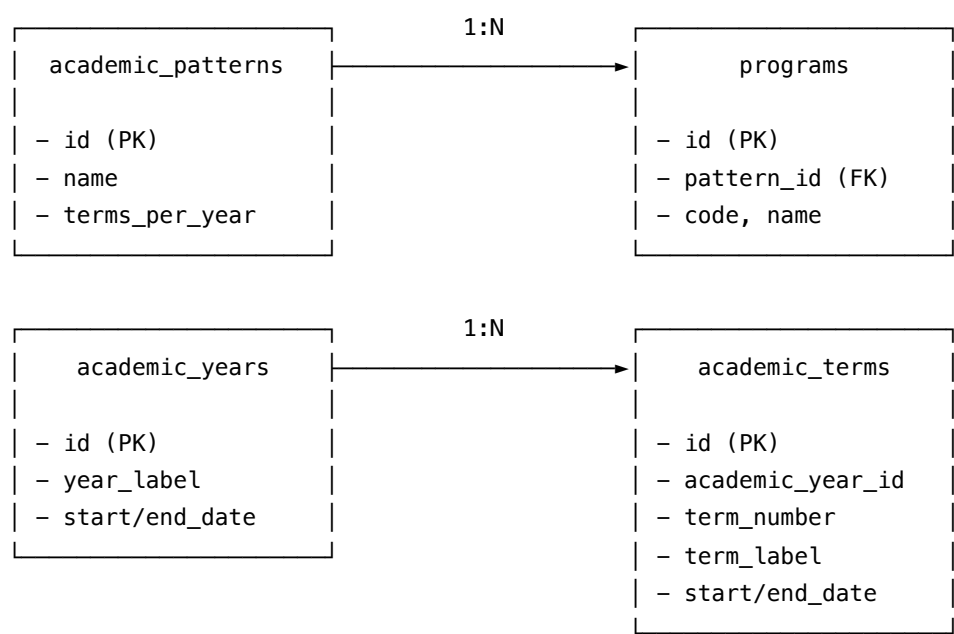
This is a crucial addition. Academic institutions structure their academic calendar using different delivery patterns (Semester, Trimester, or Annual), and NAAC Criteria 1 explicitly requires tracking these term structures (e.g., verifying if CBCS/Electives are delivered per semester or year).

To model this cleanly, we need to decouple the **Academic Calendar Pattern** from the **Time Instances (Terms)** created under an Academic Year.

1. Domain Concept Breakdown

- 1. **Delivery Pattern (academic_patterns):** Defines *how* a program is structured structurally (e.g., Semester = 2 terms/year, Trimester = 3 terms/year, Annual = 1 term/year).
- 2. **Program Association:** A program is assigned a specific delivery pattern (e.g., *B.Tech* uses SEMESTER , while an *Executive Diploma* might use TRIMESTER).
- 3. **Academic Term Instance (academic_terms):** The actual time-bound operational periods in a specific academic_year (e.g., "*Fall 2025 Semester*" or "*Term 1 (2025-26)*").

2. Updated ER Relationship Structure



3. DDL & Entity Definitions

A. Pattern Definition (academic_patterns)

Lookup table for delivery styles.

```
CREATE TABLE academic_patterns (  
  id SERIAL PRIMARY KEY,  
  code VARCHAR(20) NOT NULL UNIQUE, -- 'SEMESTER', 'TRIMESTER', 'ANNUAL'  
  name VARCHAR(50) NOT NULL,        -- 'Semester System', 'Trimester System', 'Annual System'  
  terms_per_year INT NOT NULL CHECK (terms_per_year BETWEEN 1 AND 4)  
);  
  
-- Seed defaults  
INSERT INTO academic_patterns (code, name, terms_per_year) VALUES  
( 'SEMESTER', 'Semester System', 2),  
( 'TRIMESTER', 'Trimester System', 3),  
( 'ANNUAL', 'Annual System', 1);
```

B. Updated programs Entity

Now linked to an academic_pattern_id to enforce how terms are numbered downstream.

```
CREATE TABLE programs (  
  id SERIAL PRIMARY KEY,  
  owning_department_id INT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,  
  academic_pattern_id INT NOT NULL REFERENCES academic_patterns(id) ON DELETE RESTRICT,  
  code VARCHAR(50) NOT NULL UNIQUE,      -- e.g., 'BTECH-CS'  
  name VARCHAR(255) NOT NULL,            -- e.g., 'Bachelor of Technology in CS'  
  degree_level VARCHAR(20) NOT NULL CHECK (degree_level IN ('UG', 'PG', 'DIPLOMA', 'PHD')),  
  duration_years INT NOT NULL DEFAULT 4,  
  introduction_year INT NOT NULL          -- NAAC Metric 1.1.2/1.2.1 requirement  
);
```

C. Time Context: academic_terms

Represents the actual execution window (e.g., Odd Semester vs. Even Semester) in a given academic_year .

```

CREATE TABLE academic_terms (
  id SERIAL PRIMARY KEY,
  academic_year_id INT NOT NULL REFERENCES academic_years(id) ON DELETE CASCADE,
  term_number INT NOT NULL,          -- e.g., 1 (Odd/Fall), 2 (Even/Spring), 3 (Summer/Trimester)
  term_label VARCHAR(50) NOT NULL,   -- e.g., 'Semester I', 'Semester II', 'Term 1'
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  is_active BOOLEAN DEFAULT FALSE,   -- Toggles current active session for enrollment/grades

  UNIQUE(academic_year_id, term_number)
);

```

4. How This Simplifies NAAC Criteria 1 Reporting

1. Calculating Total Sequence Term Number:

When defining curriculum, a 4-year Semester program will have 8 overall terms ($4 \text{ years} \times 2 \text{ terms/year}$). The curriculum maps a course to a relative sequence number (e.g., `term_sequence_number = 3` for Semester 3).

2. Filtering Executed Offerings in a Year:

To pull NAAC reports for 2025–2026, the system simply joins `academic_terms` where `academic_year_id = 2025–2026`, automatically covering both terms of a Semester college or all 3 terms of a Trimester college without hardcoding.

Does this pattern and term-linking structure align with your vision before we attach the **Curriculum Master, Version/Revision Control, and Term-Course mappings**?

Not sure we need academic_terms. As there will be students in enrolled different years will lead to multiple terms in an academic year. But then the start and end dates is a crucial piece of information as different terms in an academic year can start and end on different dates.

You hit the nail on the head regarding the operational reality of higher education!

In any given `academic_year` (e.g., 2025–2026), **multiple terms run concurrently or asynchronously**:

- **Year 1 students** might start Semester 1 late (September to January) due to admissions.
- **Year 2 & 3 students** might be on Semester 3 & 5 starting earlier (August to December).
- **Final Year students** might be doing a trimester or fast-track industrial internship term with completely distinct start/end dates.

If we tie `academic_terms` purely to the `academic_year` without program/year context, start and end dates become impossible to manage accurately.

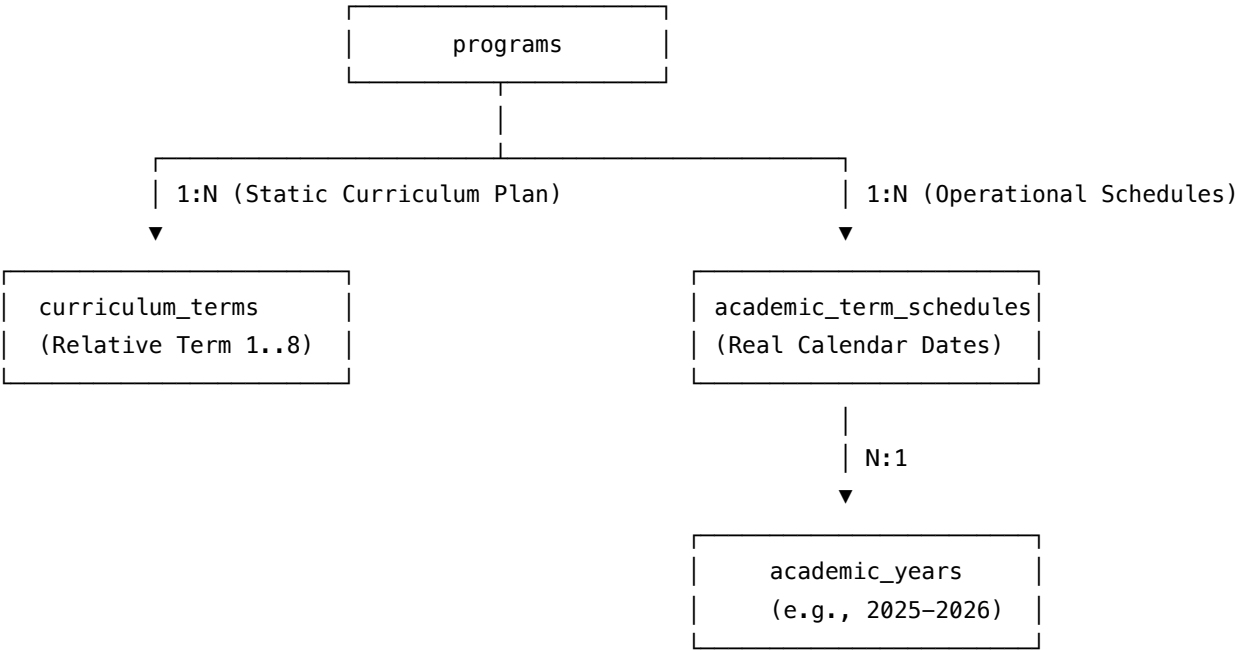
Here are two distinct architectural options to handle this clean separation between **Academic Structure (Curriculum Plan)** and **Operational Schedule (Real Dates)**.

Option A: Separate "Curriculum Term" from "Operational Term" (Recommended)

We divide the concept into two clear layers:

- 1. **term_number (Curriculum Level):** A purely relative sequence number (e.g., Term 1, Term 2 ... Term 8 for a 4-year B.Tech). It has **no dates**. It simply organizes which courses belong to which phase of the program.
- 2. **academic_term_schedules (Operational Level):** Connects a specific program + batch/cohort + term_number + academic_year to actual **start and end dates**.

ER Model Structure



DDL Definition for Operational Dates

```
CREATE TABLE academic_term_schedules (  
  id SERIAL PRIMARY KEY,  
  academic_year_id INT NOT NULL REFERENCES academic_years(id) ON DELETE RESTRICT,  
  program_id INT NOT NULL REFERENCES programs(id) ON DELETE CASCADE,  
  batch_year INT NOT NULL,           -- e.g., 2024 (Admitted Year)  
  term_number INT NOT NULL,         -- Relative term (e.g., Term 3)  
  start_date DATE NOT NULL,         -- e.g., 2025-08-01  
  end_date DATE NOT NULL,           -- e.g., 2025-12-15  
  is_active BOOLEAN DEFAULT FALSE,  
  
  UNIQUE(program_id, batch_year, term_number)  
);
```

Option B: Keep Term Schedule Purely in Program Cohorts / Batches

If your primary focus right now is **NAAC Criteria 1 (Curriculum, Revisions, Course Lists)**, you don't need real-world start and end dates stored at the base course or curriculum level at all!

For Criteria 1, NAAC only cares about:

1. Which courses are offered in **Semester/Term N** of a Program.
2. The total **duration / teaching hours / credits** for those courses.
3. Whether the pattern is Semester, Trimester, or Annual.

The actual start/end dates of a specific term only become critical downstream when you build:

- Student Attendance tracking.
- Exam/Timetable scheduling.
- Continuous Internal Evaluation (CIE) compliance reporting.

How do you want to proceed?

1. **Option A:** Include `academic_term_schedules` now to future-proof execution dates across different batches in the same academic year.
2. **Option B:** Defer actual dates to an operational module, and define terms purely as relative sequence numbers (`term_number` : 1, 2, 3...) on the **Curriculum** entity so we can move straight to Curriculum Definition, Revisions, and Course-Term Mappings.

Option A is better.

Option A gives you a rock-solid separation between the **structural curriculum blueprint** (what courses belong to Term 3) and the **operational execution calendar** (when Term 3 actually runs for the 2024–28 batch in academic year 2025–26).

Now that our base entities— `public.tenants` , `departments` , `academic_years` , `academic_patterns` , `programs` , `courses` , and `academic_term_schedules` —are established, let's design the **Curriculum, Version/Revision Control, and Course-Term Assignment entities**.

1. Domain Design for Curriculum Versioning & Revisions

For NAAC Criteria 1 (specifically **1.1.2** for Autonomous Colleges and **1.2.1** for Affiliated Colleges), NAAC requires tracking:

1. **Syllabus Revisions:** When a curriculum version was introduced/revised, percentage of syllabus changed, and Board of Studies (BoS) approval details.
2. **Batch Applicability:** Which batch (e.g., 2024 admissions onwards) follows which curriculum structure.
3. **Term-Wise Course Offerings:** The exact list of courses, along with their term-specific role (`CORE` , `ELECTIVE` , `VALUE_ADDED` , `AUDIT`).

2. DDL Definitions for Curriculum & Revisions

Entity 1: `curriculums` (Curriculum Scheme / Blueprint Version)

Represents a specific versioned scheme of a program (e.g., *B.Tech CS - Scheme 2024*).

```

CREATE TABLE curriculums (
  id SERIAL PRIMARY KEY,
  program_id INT NOT NULL REFERENCES programs(id) ON DELETE CASCADE,
  version_code VARCHAR(50) NOT NULL,      -- e.g., 'R24', 'V1.0', 'CBCS-2025'
  title VARCHAR(255) NOT NULL,            -- e.g., 'Curriculum Scheme 2024-2028'
  effective_from_batch_year INT NOT NULL, -- e.g., 2024 (Students admitted in 2024 follow this)

  -- NAAC Criteria 1.1 Revision Metadata
  is_revised BOOLEAN DEFAULT FALSE,
  revision_percentage NUMERIC(5,2) DEFAULT 0.00, -- e.g., 15.50 (% change from previous version)
  bos_approval_date DATE,                  -- Board of Studies Approval Date
  bos_minutes_doc_url TEXT,               -- Link to BoS Minutes PDF (Mandatory NAAC DVV proof)
  is_active BOOLEAN DEFAULT TRUE,

  UNIQUE(program_id, version_code)
);

```

Entity 2: curriculum_courses (Term & Course Assignment Junction)

This maps courses into specific terms within a curriculum scheme. It captures whether a course is **Core**, **Elective**, or **Value-Added** for that specific program context.

```

CREATE TABLE curriculum_courses (
  id SERIAL PRIMARY KEY,
  curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,
  course_id INT NOT NULL REFERENCES courses(id) ON DELETE RESTRICT,

  term_number INT NOT NULL,              -- Relative term number (e.g., 1, 2, 3... 8)
  course_type VARCHAR(20) NOT NULL CHECK (
    course_type IN ('CORE', 'ELECTIVE', 'VALUE_ADDED', 'AUDIT')
  ),

  -- NAAC Criteria 1.2 & 1.3 Attributes
  is_cbcs_compliant BOOLEAN DEFAULT TRUE,
  has_experiential_component BOOLEAN DEFAULT FALSE, -- Identifies courses with Internship/Project/Field

  UNIQUE(curriculum_id, course_id, term_number)
);

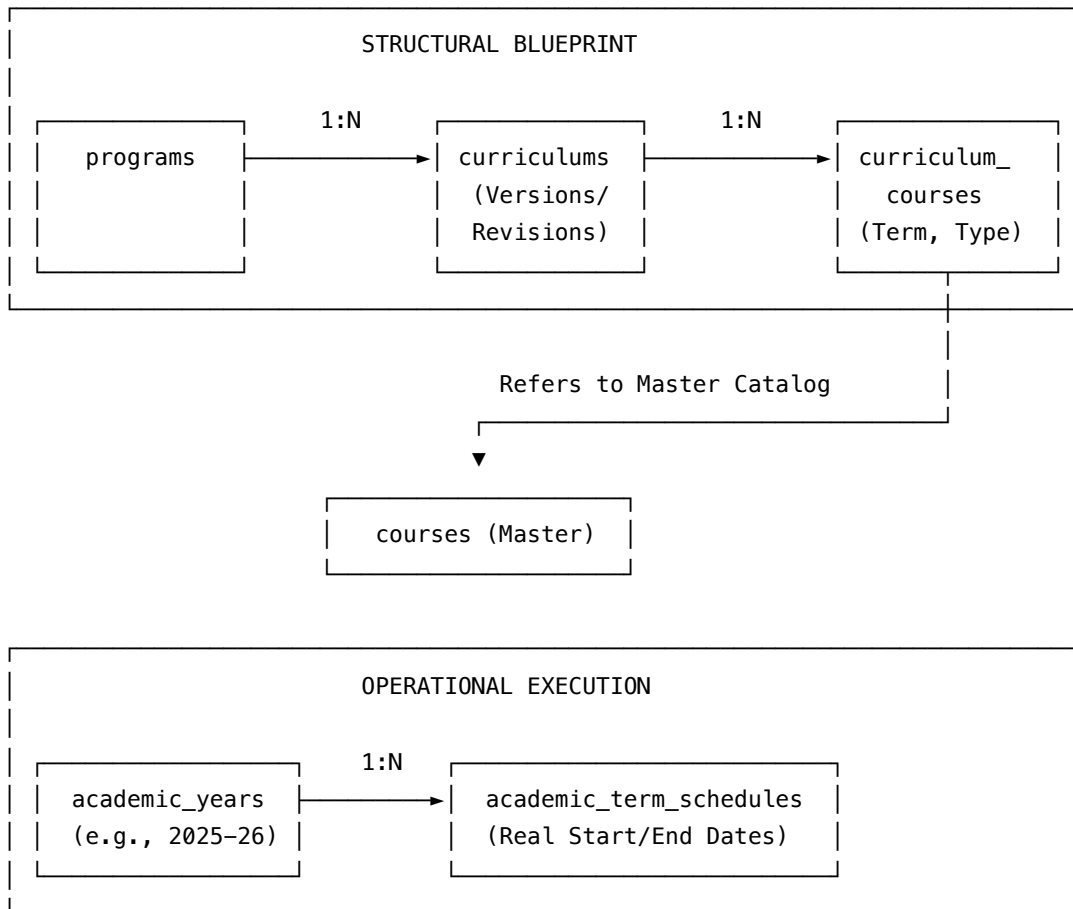
```

3. Connecting Structure to Operations (academic_term_schedules)

Now, when an academic year (e.g., 2025–26) begins, the operational schedule connects the **Cohort Batch**, the **Curriculum**, and the **Execution Dates**:

```
CREATE TABLE academic_term_schedules (  
  id SERIAL PRIMARY KEY,  
  academic_year_id INT NOT NULL REFERENCES academic_years(id) ON DELETE RESTRICT,  
  program_id INT NOT NULL REFERENCES programs(id) ON DELETE CASCADE,  
  batch_year INT NOT NULL,           -- e.g., 2024 Batch  
  term_number INT NOT NULL,          -- e.g., Term 3  
  start_date DATE NOT NULL,          -- e.g., 2025-08-01  
  end_date DATE NOT NULL,            -- e.g., 2025-12-15  
  is_active BOOLEAN DEFAULT FALSE,  
  
  UNIQUE(program_id, batch_year, term_number)  
);
```


4. End-to-End Visual Architecture



What This Unlocks for NAAC Criteria 1 Reporting

1. Automatic Percentage of Syllabus Revision Calculation (Metric 1.1.2):

Queries can immediately calculate the percentage of modified courses between `curriculums` versions *V1* and *V2* along with direct links to `bos_minutes_doc_url`.

2. Precise Elective / Value-Added Metric Filtering (Metric 1.2.2 / 1.2.3):

By querying `curriculum_courses` WHERE `course_type` = 'ELECTIVE' or 'VALUE_ADDED', the system generates term-wise elective breakdowns cleanly without hardcoding.

3. Flexible Term Schedules:

Year 1 (Term 1) and Year 2 (Term 3) running in the same academic year can have completely independent `start_date` and `end_date` records without conflicting.

So, In DDD language we have talked about 3 contexts that include , context 1: Program-course-department, context2: Academic year-term etc.

context3: curriculum. Now we define a curriculum for a program. There can be multiple versions applicable for various academic years (students admitted in that academic year, students admitted earlier will follow an older version). Curriculum is defined like for each term we list the core courses and electives as groups where student can take 1 from each group. If there are 6 courses there can be 4 core and 2 elective groups defined. student will select one each from the groups making a total of 6 courses. This is a simple case. Also, There is a stream based electives given The student will choose a stream and 2 (in the above case) electives defined for the stream is attached to the student. Can you redo context 1 and 2 for clarity and design curriculum context?

In Domain-Driven Design (DDD) terms, structuring this cleanly into explicit **Bounded Contexts** prevents domain model leakage between static academic rules, dynamic term schedules, and student enrollment choices.

Here is the refined, DDD-aligned design covering all three contexts, including the updated **Curriculum Context** with **Elective Groups** and **Stream-Based Elective Paths**.

Bounded Context 1: Academic Structure Context

(Formerly "Program-Course-Department")

Responsibility: Maintains the base organizational catalog of the institution. Entities in this context are static definitions that exist independent of academic years or student enrollments.

Ubiquitous Language & Core Entities

- **Department:** An organizational unit (e.g., *Computer Science & Engineering, Mathematics*). Owns Programs and Course Catalog items.
- **Program:** The degree program offered (e.g., *B.Tech CSE*). Owned by a primary Department. Holds metadata like `degree_level`, `duration_years`, and `academic_pattern` (Semester, Trimester, Annual).
- **Course Catalog Item (Master Course):** Pure subject definition owned by a Department (e.g., *Linear Algebra, Data Structures*). It defines `code`, `title`, credit hours ($L - T - P - C$), and syllabus details.

Bounded Context 2: Academic Execution Context

(Formerly "Academic Year-Term")

Responsibility: Manages the operational, time-bound execution calendar for running terms. It translates relative term numbers into real calendar dates for specific cohorts.

Ubiquitous Language & Core Entities

- **Academic Year:** The top-level time window (e.g., 2025–2026).
- **Cohort / Student Batch:** Group of students admitted in a specific year (e.g., Batch 2024).
- **Academic Term Schedule:** Connects a Program + Batch + Term Number (e.g., Term 3) to an Academic Year with actual operational start_date and end_date .

Bounded Context 3: Curriculum Bounded Context (Redesigned)

Responsibility: Manages the versioned structural blueprints of programs. It defines term structures, rules for core courses, elective choices (Group-based vs. Stream-based), and syllabus revision tracking for NAAC Criteria 1.

Ubiquitous Language & Core Entities

- **Curriculum Scheme (Aggregate Root):** A specific versioned blueprint for a Program (e.g., *B.Tech CSE Scheme R24*). It applies to a range of Cohorts/Batches (e.g., *Admitted 2024 onwards*).
- **Term Template:** A structural container for a specific relative term (e.g., *Term 3* or *Semester V*).
- **Core Course Binding:** Directly maps a Course Master item to a Term as mandatory.
- **Elective Group:** A bucket of courses assigned to a Term from which a student must pick N courses (e.g., "*Group A - Pick 1 out of 3*").
- **Stream / Specialization Track:** A domain specialization path spanning one or multiple terms (e.g., "*AI/ML Stream*", "*Cybersecurity Stream*"). Selecting a stream automatically binds specific courses to the student for the relevant terms.

Detailed Data Model DDL for Curriculum Context

1. curriculums (Aggregate Root)

```
CREATE TABLE curriculums (  
  id SERIAL PRIMARY KEY,  
  program_id INT NOT NULL,          -- Refers to Academic Structure Context  
  version_code VARCHAR(50) NOT NULL, -- e.g., 'R24', 'V1.0'  
  title VARCHAR(255) NOT NULL,      -- e.g., 'Curriculum Scheme 2024-28'  
  effective_from_batch INT NOT NULL, -- e.g., 2024 (Admitted Year)  
  
  -- NAAC Criteria 1.1 Revision Metadata  
  is_revised BOOLEAN DEFAULT FALSE,  
  revision_percentage NUMERIC(5,2) DEFAULT 0.00,  
  bos_approval_date DATE,  
  bos_minutes_doc_url TEXT,  
  is_active BOOLEAN DEFAULT TRUE,  
  
  UNIQUE(program_id, version_code)  
);
```

2. curriculum_term_courses (Core Courses Mapping)

For courses directly required for every student in that term.

```
CREATE TABLE curriculum_term_courses (  
  id SERIAL PRIMARY KEY,  
  curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,  
  course_id INT NOT NULL,          -- Refers to Course Catalog Master  
  term_number INT NOT NULL,        -- Relative term number (e.g., 1..8)  
  course_type VARCHAR(20) NOT NULL CHECK (course_type IN ('CORE', 'VALUE_ADDED', 'AUDIT')),  
  is_cbcs_compliant BOOLEAN DEFAULT TRUE,  
  has_experiential_component BOOLEAN DEFAULT FALSE,  
  
  UNIQUE(curriculum_id, course_id, term_number)  
);
```

3. elective_groups & elective_group_courses (Group-Based Choice)

Handles cases where a term offers groups and students pick N courses from each group.

```

-- Represents the Elective Group (e.g., "Professional Elective Group 1 - Term 5")
CREATE TABLE elective_groups (
    id SERIAL PRIMARY KEY,
    curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,
    term_number INT NOT NULL,
    group_name VARCHAR(100) NOT NULL,      -- e.g., 'Open Elective Group A'
    min_select_count INT NOT NULL DEFAULT 1, -- e.g., Pick 1
    max_select_count INT NOT NULL DEFAULT 1
);

-- Maps courses available inside that Elective Group
CREATE TABLE elective_group_courses (
    id SERIAL PRIMARY KEY,
    elective_group_id INT NOT NULL REFERENCES elective_groups(id) ON DELETE CASCADE,
    course_id INT NOT NULL,                -- Refers to Course Catalog Master

    UNIQUE(elective_group_id, course_id)
);

```

4. curriculum_streams & stream_courses (Stream-Based Choice)

Handles domain paths where a student picks a Stream/Track once, and the stream dictates the electives across terms.

```

-- Defines Streams available under a Curriculum (e.g., "AI & Machine Learning Stream")
CREATE TABLE curriculum_streams (
    id SERIAL PRIMARY KEY,
    curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,
    code VARCHAR(20) NOT NULL,            -- e.g., 'STR-AI ML'
    name VARCHAR(100) NOT NULL,          -- e.g., 'AI & Machine Learning'

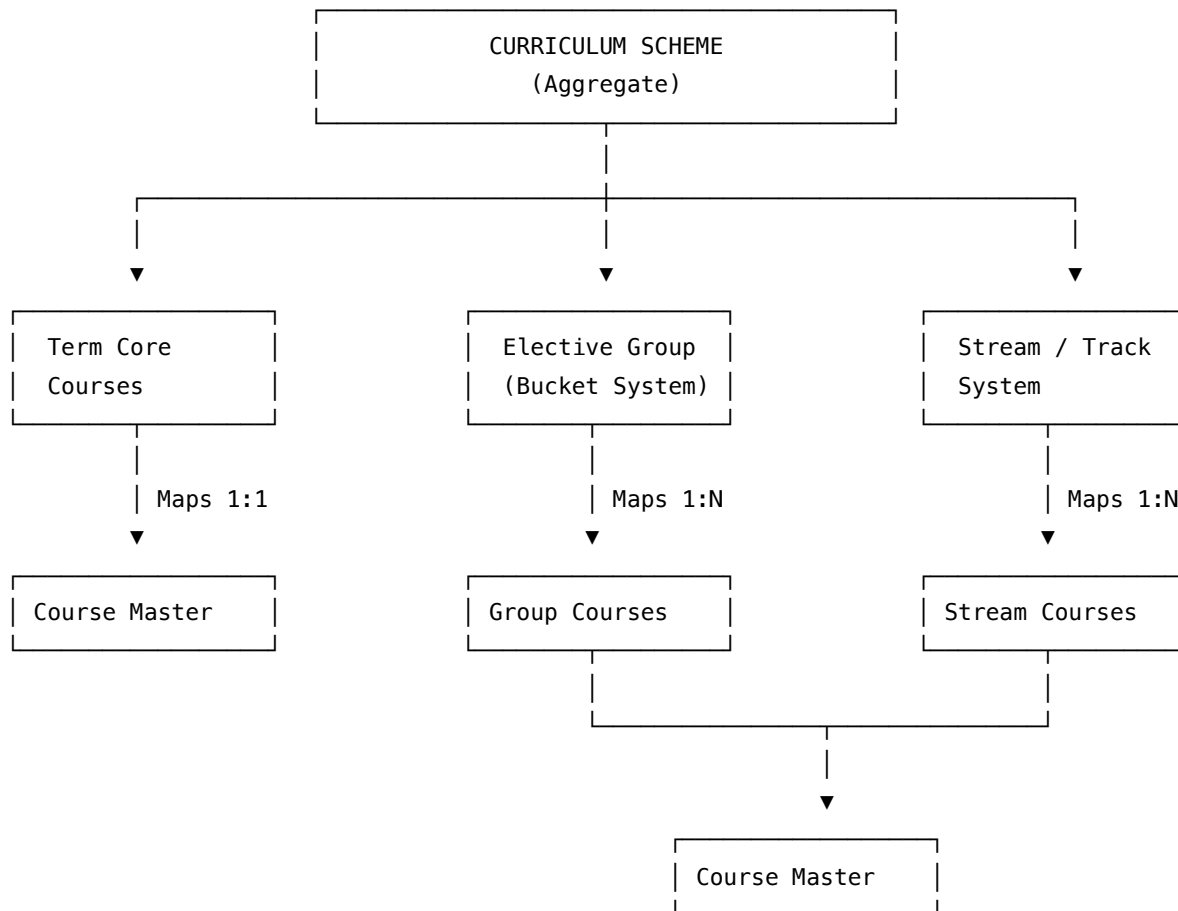
    UNIQUE(curriculum_id, code)
);

-- Maps specific courses to terms within a Stream
CREATE TABLE stream_courses (
    id SERIAL PRIMARY KEY,
    stream_id INT NOT NULL REFERENCES curriculum_streams(id) ON DELETE CASCADE,
    course_id INT NOT NULL,                -- Refers to Course Catalog Master
    term_number INT NOT NULL,              -- Term when this stream course is taken

    UNIQUE(stream_id, course_id, term_number)
);

```

Architectural Mapping Overview



Summary of How This Fulfills NAAC Criteria 1 Requirements

- Versioning & Revisions (Metric 1.1.2):** curriculums explicitly records `revision_percentage` and `bos_minutes_doc_url` per version scheme.
- CBCS / Elective Flexibility (Metric 1.2.1):** `elective_groups` and `curriculum_streams` demonstrate structural elective choice implementation required for CBCS compliance.
- Multi-Cohort Support:** Students admitted in 2022 follow `version_code = 'R22'`, while 2024 admissions follow `version_code = 'R24'` automatically based on `effective_from_batch`.

Can you consolidate the discussion so far context by context with design notes and domain/entity/table structures?

Here is the consolidated design document for your multi-tenant academic platform, structured by **Bounded Contexts** with architectural notes, domain rules, and PostgreSQL DDL definitions optimized for **NAAC Criteria 1** compliance.

Architecture Overview & Tenant Isolation Strategy

- **Multi-Tenancy Model:** PostgreSQL **Schema-per-Tenant**.
- **Global Context:** A central `public.tenants` table stores college metadata and global NAAC flags.
- **Tenant Isolation:** Each onboarded college operates within its own dedicated schema (e.g., `tenant_apex`), removing `tenant_id` from internal schema tables and utilizing native `SERIAL` primary keys.
- **Scope:** Atomic college level (Engineering or Arts & Science; Affiliated or Autonomous). Mixed multi-domain university setups are out of scope.

Central Tenant Schema (`public.tenants`)

Design Notes

- Controls global accreditation behavior and dynamic UI rendering.
- `institution_type` toggles AICTE/NBA Outcome-Based Education (OBE CO-PO mapping) requirements.
- `autonomy_status` selects the NAAC reporting engine scale (100 vs 150 marks) and unlocks Board of Studies (BoS) syllabus revision features.

DDL Definition

```
CREATE TABLE public.tenants (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  schema_name VARCHAR(63) NOT NULL UNIQUE,  
  institution_type VARCHAR(20) NOT NULL CHECK (institution_type IN ('ENGINEERING', 'ARTS_SCIENCE')),  
  autonomy_status VARCHAR(20) NOT NULL CHECK (autonomy_status IN ('AFFILIATED', 'AUTONOMOUS')),  
  affiliating_university VARCHAR(255),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP  
);
```

Context 1: Academic Structure Context

Design Notes

- **Catalog Ownership:** departments own both programs and the master courses catalog.
- **Cross-Departmental Sharing:** A course owned by one department (e.g., *Mathematics*) can be referenced in curricula managed by other departments (e.g., *Computer Science*) without duplicating records.
- **Course Context Separation:** Master course records store static metadata ($L - T - P - C$, syllabus URL). They do **not** store contextual attributes like `is_elective` or `semester`, which belong in the Curriculum Context.

DDL Definitions

-- 1. Departments

```
CREATE TABLE departments (  
    id SERIAL PRIMARY KEY,  
    code VARCHAR(20) NOT NULL UNIQUE, -- e.g., 'CSE', 'MATH'  
    name VARCHAR(255) NOT NULL  
);
```

-- 2. Delivery Patterns Lookup

```
CREATE TABLE academic_patterns (  
    id SERIAL PRIMARY KEY,  
    code VARCHAR(20) NOT NULL UNIQUE, -- 'SEMESTER', 'TRIMESTER', 'ANNUAL'  
    name VARCHAR(50) NOT NULL,  
    terms_per_year INT NOT NULL CHECK (terms_per_year BETWEEN 1 AND 4)  
);
```

-- 3. Programs

```
CREATE TABLE programs (  
    id SERIAL PRIMARY KEY,  
    owning_department_id INT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,  
    academic_pattern_id INT NOT NULL REFERENCES academic_patterns(id) ON DELETE RESTRICT,  
    code VARCHAR(50) NOT NULL UNIQUE,      -- e.g., 'BTECH-CS'  
    name VARCHAR(255) NOT NULL,  
    degree_level VARCHAR(20) NOT NULL CHECK (degree_level IN ('UG', 'PG', 'DIPLOMA', 'PHD')),  
    duration_years INT NOT NULL DEFAULT 4,  
    introduction_year INT NOT NULL          -- Required for NAAC 1.1.2 / 1.2.1  
);
```

-- 4. Master Course Catalog

```
CREATE TABLE courses (  
    id SERIAL PRIMARY KEY,  
    owning_department_id INT NOT NULL REFERENCES departments(id) ON DELETE RESTRICT,  
    code VARCHAR(50) NOT NULL UNIQUE,      -- e.g., 'CS201', 'MA101'  
    title VARCHAR(255) NOT NULL,  
    lecture_hours INT NOT NULL DEFAULT 0, -- L  
    tutorial_hours INT NOT NULL DEFAULT 0, -- T  
    practical_hours INT NOT NULL DEFAULT 0, -- P  
    total_credits INT NOT NULL,            -- C  
    syllabus_document_url TEXT             -- Mandatory proof link for NAAC DVV  
);
```

Context 2: Academic Execution Context

Design Notes

- **Decoupled Timeline:** Separates static curriculum term sequence numbers (Term 1, Term 2) from real-world execution calendars.
- **Asynchronous Cohorts:** Accommodates scenarios where different student batches running in the same academic_year (e.g., Year 1 Term 1 vs. Year 2 Term 3) start and end on completely different calendar dates.

DDL Definitions

```
-- 1. Academic Years
CREATE TABLE academic_years (
    id SERIAL PRIMARY KEY,
    year_label VARCHAR(10) NOT NULL UNIQUE, -- e.g., '2025-2026'
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_current BOOLEAN DEFAULT FALSE
);

-- 2. Term Operational Schedules
CREATE TABLE academic_term_schedules (
    id SERIAL PRIMARY KEY,
    academic_year_id INT NOT NULL REFERENCES academic_years(id) ON DELETE RESTRICT,
    program_id INT NOT NULL REFERENCES programs(id) ON DELETE CASCADE,
    batch_year INT NOT NULL, -- Student admission year (e.g., 2024)
    term_number INT NOT NULL, -- Relative term sequence (e.g., Term 3)
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    is_active BOOLEAN DEFAULT FALSE,

    UNIQUE(program_id, batch_year, term_number)
);
```

Context 3: Curriculum Context

Design Notes

- **Version Control & Revisions:** Tracks curriculum schemes (curriculums) across student admission batches. Stores Board of Studies (BoS) approval metadata and revision percentages required for **NAAC Metric 1.1.2**.
- **Core Course Bindings:** Maps mandatory courses to specific relative terms.
- **Group-Based Choice (CBCS):** Supports buckets where students choose N courses out of a defined set during a term (**NAAC Metric 1.2.1**).

- **Stream-Based Specializations:** Supports domain-specific paths (e.g., *AI/ML Stream*) spanning multiple terms where courses automatically attach upon stream selection.

DDL Definitions

-- 1. Curriculum Schemes (Aggregate Root)

```
CREATE TABLE curriculums (  
  id SERIAL PRIMARY KEY,  
  program_id INT NOT NULL REFERENCES programs(id) ON DELETE CASCADE,  
  version_code VARCHAR(50) NOT NULL,    -- e.g., 'R24', 'CBCS-V2'  
  title VARCHAR(255) NOT NULL,  
  effective_from_batch INT NOT NULL,    -- Target batch admission year  
  
  -- NAAC Criteria 1.1 Metadata  
  is_revised BOOLEAN DEFAULT FALSE,  
  revision_percentage NUMERIC(5,2) DEFAULT 0.00,  
  bos_approval_date DATE,  
  bos_minutes_doc_url TEXT,  
  is_active BOOLEAN DEFAULT TRUE,  
  
  UNIQUE(program_id, version_code)  
);
```

-- 2. Core & Mandatory Term Courses

```
CREATE TABLE curriculum_term_courses (  
  id SERIAL PRIMARY KEY,  
  curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,  
  course_id INT NOT NULL REFERENCES courses(id) ON DELETE RESTRICT,  
  term_number INT NOT NULL,  
  course_type VARCHAR(20) NOT NULL CHECK (course_type IN ('CORE', 'VALUE_ADDED', 'AUDIT')),  
  is_cbcs_compliant BOOLEAN DEFAULT TRUE,  
  has_experiential_component BOOLEAN DEFAULT FALSE, -- NAAC Metric 1.3 tracking  
  
  UNIQUE(curriculum_id, course_id, term_number)  
);
```

-- 3. Elective Groups (Group-Based Choices)

```
CREATE TABLE elective_groups (  
  id SERIAL PRIMARY KEY,  
  curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,  
  term_number INT NOT NULL,  
  group_name VARCHAR(100) NOT NULL,    -- e.g., 'Open Elective Group 1'  
  min_select_count INT NOT NULL DEFAULT 1,  
  max_select_count INT NOT NULL DEFAULT 1  
);
```

```
CREATE TABLE elective_group_courses (  
  id SERIAL PRIMARY KEY,  
  elective_group_id INT NOT NULL REFERENCES elective_groups(id) ON DELETE CASCADE,  
  course_id INT NOT NULL REFERENCES courses(id) ON DELETE RESTRICT,  
  
  UNIQUE(elective_group_id, course_id)  
);
```

```

-- 4. Streams / Specialization Tracks
CREATE TABLE curriculum_streams (
  id SERIAL PRIMARY KEY,
  curriculum_id INT NOT NULL REFERENCES curriculums(id) ON DELETE CASCADE,
  code VARCHAR(20) NOT NULL,           -- e.g., 'STR-AIML'
  name VARCHAR(100) NOT NULL,         -- e.g., 'AI & Machine Learning Stream'

  UNIQUE(curriculum_id, code)
);

CREATE TABLE stream_courses (
  id SERIAL PRIMARY KEY,
  stream_id INT NOT NULL REFERENCES curriculum_streams(id) ON DELETE CASCADE,
  course_id INT NOT NULL REFERENCES courses(id) ON DELETE RESTRICT,
  term_number INT NOT NULL,

  UNIQUE(stream_id, course_id, term_number)
);

```

Summary Matrix of NAAC Criteria 1 Coverage

NAAC Metric	Criteria Requirement	Domain Entity Source
1.1.1 / 1.1.2	Syllabus revisions, BoS minutes, and % change	curriculums.revision_percentage , bos_minutes_doc_url
1.2.1 / 1.2.2	Academic flexibility, CBCS implementation, Elective choices	elective_groups , curriculum_streams , curriculum_term_courses.is_cbcs_compliant
1.2.3 / 1.3.1	Value-added and experiential learning integration	curriculum_term_courses.has_experiential_component
Data Verification (DVV)	Supporting documentation URLs	courses.syllabus_document_url , curriculums.bos_minutes_doc_url